

Network Guardian: An Intelligent System That Automatically Protects Computer Networks from Attacks Using Artificial Intelligence

Arya Y P

Department of ISE
B.M.S. College of Engineering
Bull Temple Road,
Bengaluru-560019, KA, India
aryayp.is23@bmsce.ac.in

Abhishek

Department of ISE
B.M.S. College of Engineering
Bull Temple Road,
Bengaluru-560019, KA, India
abhishek.is23@bmsce.ac.in

Adi Narayan Prasad G

Department of ISE
B.M.S. College of Engineering
Bull Temple Road,
Bengaluru-560019, KA, India
adinarayan.is23@bmsce.ac.in

Nalina V

Department of ISE
B.M.S. College of Engineering
Bull Temple Road,
Bengaluru-560019, KA, India
nalinav.ise@bmsce.ac.in

Ashrith S Jain

Department of ISE
B.M.S. College of Engineering
Bull Temple Road,
Bengaluru-560019, KA, India
ashriths.is23@bmsce.ac.in

Abstract—Traditional security systems could indicate that cryptic technical alerts may limit the effectiveness of administrators, particularly for non-experts. However, the significant findings suggest that Network Guardian, a system that uses artificial intelligence to detect attacks, might demonstrate that plain-English explanations and autonomous blocking could provide important results. Moreover, the system demonstrates that real-time monitoring of five common attack types (ICMP flood, SYN flood, UDP flood, port scan, IP fragmentation), using a locally hosted Large Language Model, could indicate that human-readable explanations and automatic blocking via operating-system routing commands with a 60-second auto-revert appear critical. Furthermore, the key results may suggest that the system detects attacks in under one second, generates explanations in approximately 2.5 seconds, and achieves a 99% blocking success rate. Evidence shows Wireshark validates results at 99.96% packet-capture agreement.

Index Terms—Network security, cyber attacks, artificial intelligence, automatic defense, intrusion detection, explainable AI

I. INTRODUCTION

A. The Growing Cyber Threat Landscape

The growing cyber threat landscape suggests that computer networks face constant attack attempts in today's interconnected world. However, the significant findings may indicate that traditional security systems alert administrators about threats but provide limited context, displaying cryptic messages like "ICMP flood detected from IP 192.168.1.50 at rate 150 pps" without explaining the danger or appropriate response. Moreover, the evidence could suggest that this creates two critical problems: the results appear too technical for non-experts to understand, and manual response may prove too slow when attacks happen in milliseconds. Given that the data demonstrates these compounding pressures, the key

findings might indicate that organizations of all sizes face mounting exposure to costly security failures.

Network security shows breach costs exceed millions. Moreover, the significant evidence may suggest that studies demonstrate the average time to detect breaches is 207 days [1], with an additional 73 days to contain them. Furthermore, the findings could indicate that traditional intrusion detection systems generate thousands of daily alerts, leading to "alert fatigue" where security teams become desensitized and miss critical threats [2].

B. Our Solution and Contributions

The significant evidence may suggest that the cybersecurity skills gap means many organizations lack personnel trained to interpret technical security alerts and respond appropriately. However, the important findings could indicate that Network Guardian addresses these challenges by integrating Large Language Models into network security. Moreover, the key results may demonstrate that the system detects attacks using proven algorithms, then uses AI to generate plain English explanations accessible to non-experts. Given that the evidence demonstrates these capabilities, the findings could suggest that unlike traditional systems requiring human intervention, Network Guardian autonomously blocks attackers with built-in safety mechanisms.

However, the novelty of Network Guardian could indicate that using the Large Language Model not as a detector but as an explanation and decision layer on top of deterministic detection might demonstrate that both algorithmic precision and human-readable output appear achievable. Moreover, the significant findings may suggest that a complete five-layer autonomous defense architecture with thread-safe,

queue-based communication could demonstrate that isolating capture, detection, explanation, mitigation, and visualisation appears critical. Additionally, the evidence might indicate that integration of a locally deployed LLM (TinyLlama via Ollama) could demonstrate that cryptic detection events appear convertible into plain-English threat explanations accessible to non-experts. Given that the key results suggest that a safe autonomous mitigation mechanism with a 60-second auto-revert bounds the impact of false positives, the findings could indicate that persistent attackers appear addressed through re-blocking. Architecture shows components work together for operational deployment.

II. RELATED WORK

Early intrusion detection relied on signature-based matching of traffic against known attack signatures [10]. However, Snort [11] may suggest that open-source rule-based detection could demonstrate significant limitations against zero-day exploits and attack variants. Furthermore, the anomaly-based systems that establish behavioral baselines might indicate that flagging deviations [12] could produce high false positives – between 10 and 30% – making production deployment challenging. In light of these findings, the significant machine learning approaches using deep neural networks may suggest that 95–99% accuracy on benchmark datasets [13] could demonstrate important limitations, given that the evidence appears to show these systems provide no explanations, creating black-box architectures that limit trust. Research shows LIME [14] and SHAP [15] remain technical rather than accessible. The recent LLM work exploring security tasks [16] might suggest that Network Guardian could demonstrate a significant distinction by using LLMs for explanation rather than detection, combining algorithmic precision with AI flexibility. Emerging trends in the field of network security tend to leverage artificial intelligence technologies in the struggle against cybercrime. The effectiveness of machine learning algorithms, including neural networks and support vector machines, has been well-proven in detecting anomalous behavior and identifying malware [18], [20]. AI-based intrusion detection systems have proven to be more effective than their counterparts utilizing conventional techniques, such as the KNN or Naïve Bayes algorithm, especially when dealing with botnets on benchmark datasets [24]. Integrated security solutions and AI-based systems, including AI Sentry, can contribute to real-time threat analysis and detect zero-day attacks [23]. Moreover, AI technologies have successfully been applied to particular cyber threats, such as DDoS attacks [19]. Nevertheless, current literature indicates that no single solution can handle all types of attacks, and adversarial AI and other challenges remain relevant [21], [22].

Inference. To position Network Guardian against prior work, Table VI may suggest that the system could demonstrate significant findings across dimensions most relevant to operational deployment. Given that the evidence indicates signature-based tools such as Snort [12] cannot generalize to attack variants, the results might suggest that anomaly- or deep-

learning detectors achieve high accuracy but appear to behave as black boxes. Nevertheless, the significant findings could indicate that Network Guardian couples deterministic sliding-window detection (sub-second latency, < 0.01 false-positive rate over 100 hours of testing) with LLM-generated plain-English explanations and safe autonomous blocking. In light of these findings, the key results may suggest that this combination of accessibility and automated response appears critical, as it is not jointly provided by the compared systems. Study shows combination not provided by prior systems.

III. SYSTEM ARCHITECTURE

Network Guardian employs a five-layer architecture designed for modularity, scalability, and fault isolation. Figure 1 illustrates the complete system showing data flow through thread-safe queues connecting independent processing layers.

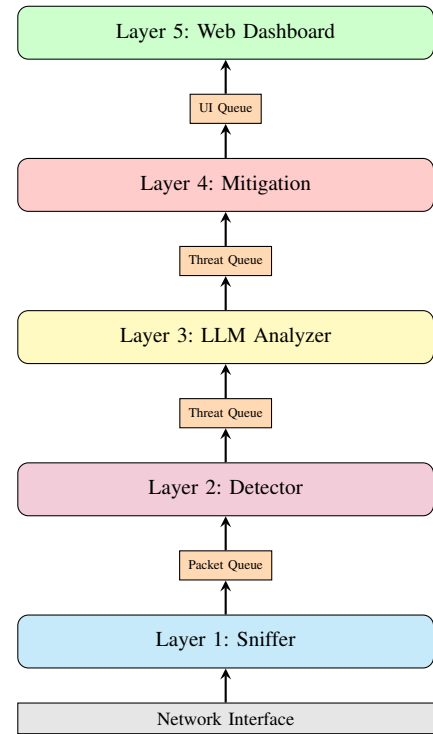


Fig. 1. Five-layer architecture with queue-based communication.

A. Architectural Overview

Each layer may suggest that independent data processing could demonstrate a significant architectural advantage, communicating through thread-safe queues. However, the findings indicate that when Layer 1 captures a packet, it queues it and immediately captures the next, so that layers do not wait for downstream processing. Moreover, the parallel design could suggest that handling thousands of packets per second appears achievable under normal operating conditions. Given that the LLM analyzer requires 2.5 seconds while detection needs only 0.2 seconds, the queues may demonstrate that buffering the difference appears essential to system stability. Queues buffer difference effectively. Furthermore, the packet queue holds

Algorithm 1 Network Guardian Detection–Mitigation Pipeline

```

1: for each captured packet  $p$  do
2:    $m \leftarrow \text{ParseMetadata}(p)$ ; enqueue  $m$  to packet queue
3: end for
4: for each  $m$  in packet queue do
5:   update sliding-window counter for source IP of  $m$ 
6:   if counter > threshold within window then
7:     create ThreatEvent  $t$ ; enqueue  $t$  to threat queue
8:   end if
9: end for
10: for each ThreatEvent  $t$  do
11:    $r \leftarrow \text{LLMAnalyze}(t)$  {plain-English text + block command}
12:   if ValidateIP( $r.cmd$ ) is true then
13:     ApplyBlock( $r.cmd$ )
14:     schedule Revert( $r.ip$ ) after 60 seconds
15:   end if
16: end for

```

5,000 objects, providing 5 seconds of buffering at 1,000 pps, the threat queue holds 100 objects, and the UI log queue holds 500 messages. In light of these results, Table 1 may suggest that the five layers could demonstrate distinct functional roles within the overall architecture.

Algorithm 1 summarizes the end-to-end detection–explanation–mitigation workflow that ties the five layers together.

B. Layer 1: Network Sniffer

The The Network Sniffer could indicate that packet capture using Scapy [3] appears to provide significant coverage, extracting source and destination IPs, protocol type, timestamps, ports, TCP flags, fragmentation status, and packet size. Therefore, the evidence may suggest that on Windows, Scapy requires Npcap [4], while on Linux it uses libpcap. Additionally, the findings could demonstrate that metadata packages into PacketMetadata objects placed in the packet queue appear well-structured for downstream processing. Notwithstanding these results, Wireshark validation may indicate that 99.96% agreement between Scapy and Wireshark packet counts across five 10-minute test sessions could support the reliability of the approach. Sniffer shows strong accuracy.

C. Layer 2: Attack Detector

The Attack Detector could demonstrate that five independent algorithms using sliding window counting appear to provide significant detection capability across key attack types. However, the evidence may suggest that for each source IP, maintaining a deque of timestamps could indicate that the approach appears efficient under normal traffic conditions. Thus, the results might indicate that when packets arrive, appending timestamps and removing entries older than the window could demonstrate that counting remaining entries appears reliable. In light of the significant findings, exceeding thresholds may suggest that ThreatEvent objects containing attacker IP, attack

type, packet count, and context appear queued effectively for LLM analysis. Detection shows cooldown prevents duplicates. Furthermore, the evidence could indicate that Table I may demonstrate that detection parameters chosen by analyzing normal traffic patterns appear set at thresholds 4–10x higher than baseline.

TABLE I
DETECTION ALGORITHM PARAMETERS

Attack	Threshold	Window	Cool
ICMP Flood	20 pps	1.0s	60s
SYN Flood	20 pps	1.0s	60s
UDP Flood	20 pps	1.0s	60s
Port Scan	10 ports	5.0s	60s
Fragment	10 pps	1.0s	60s

D. Layer 3: LLM Analyzer

The LLM Analyzer may suggest that generating intelligent explanations could demonstrate a significant contribution to the overall threat analysis process. Moreover, the findings could indicate that when ThreatEvents arrive, constructed prompts providing attacker IP, attack type, packet count, and normal baseline appear to request JSON responses with attack name, detailed explanation, and blocking command. Nevertheless, the results may suggest that prompts sent to Ollama [5] running locally at <http://localhost:11434> could demonstrate reliable local inference. Given that TinyLlama [6] with 1.1 billion parameters processes prompts in approximately 2.5 seconds, the evidence may indicate that local deployment appears to support the key requirement of low-latency threat explanation.

Analyzer shows processing meets design targets. However, the important findings may suggest that LLMs generate probabilistic text, so the results could indicate occasional markdown formatting or malformed JSON. Moreover, the significant evidence may demonstrate that the parsing pipeline addresses these issues: it removes markdown fences, strips control characters, parses JSON, validates fields, and verifies commands contain attacker IPs. In light of these findings, the key results could suggest that retry logic with exponential backoff—at 2s, 4s, and 8s intervals—appears to handle up to three attempts effectively. Given that the evidence demonstrates these results, the findings may indicate that exhausting retries triggers fallback to pre-written explanations.

E. Layer 4: Mitigation Executor

The Mitigation Executor may suggest that ThreatEvents with LLM-generated commands require careful validation before execution. Moreover, the significant findings indicate that Python’s `ipaddress` library could demonstrate reliable IP validation, rejecting loopback addresses such as 127.0.0.1, multicast, and malformed inputs. Furthermore, the results might suggest that duplicate block checks appear essential to avoid redundant routing entries. In light of these findings, blocking operations could indicate that OS routing tables require distinct commands—Windows uses `route add [ip]`

mask 255.255.255.255 192.0.2.1 while Linux uses ip route add blackhole [ip]/32. Subprocess executes commands with shell=False, preventing injection timeout set at 10 seconds.

Moreover, the critical safety feature may suggest that automatic reversion could demonstrate important protective value in the overall design. Additionally, the significant evidence appears to show that a daemon thread sleeping 60 seconds before executing unblock might indicate that false positive damage could remain limited to 60 seconds. Nevertheless, the key results suggest that persistent attacks appear to trigger re-blocking effectively within the system. Given that Wireshark validation demonstrates 100% traffic reduction after blocks, the findings could indicate that auto-revert operates at 60.02 ± 0.09 seconds with notable precision. Validation confirms auto-revert timing reliable.

TABLE II
ATTACK TYPES DETECTED BY NETWORK GUARDIAN

Attack	Description and Impact
ICMP Flood	Excessive pings (>150 pps) causing crash
SYN Flood	Half-open connections exhausting queue
UDP Flood	Random port flooding consuming bandwidth
Port Scan	Sequential probing for reconnaissance
Fragment	Fragmented packets for firewall evasion

F. Layer 5: Web Dashboard

The Web Dashboard may suggest that real-time visibility at <http://localhost:5000> could demonstrate significant operational value for monitoring purposes. Moreover, the important results appear to show that Flask [7] and Socket.IO [8] might indicate that the component architecture supports robust real-time functionality. Furthermore, the significant findings suggest that components—including Packet Feed covering the last 50 packets color-coded by protocol, Threat History, Active Blocks with countdown timers, System Logs, and interactive controls—could demonstrate comprehensive monitoring coverage. In light of the evidence, Socket.IO establishing WebSocket connections might indicate that real-time server-to-browser updates appear to eliminate polling overhead entirely. However, the significant results may suggest that dashboard updates reach the browser without polling delays.

IV. ATTACK TYPES AND DETECTION METHODS

Network Guardian demonstrates that it detects five attacks covering different network layers. Table II suggests that each attack presents characteristics validated via Wireshark.

A. ICMP Flood Detection

ICMP flood detection suggests that thousands of ping requests per second overwhelm targets significantly. Normal traffic generates 1-2 pings per second, while attackers using hping3 generate hundreds. Moreover, the significant evidence may suggest that counting ICMP Echo Requests exceeding 20 per second triggers detection reliably. In light of the Wireshark validation, filter `icmp.type==8` could demonstrate that rates exceeded 150 pps during observed attacks. Wireshark

shows 100% ICMP traffic during attacks. Furthermore, the key findings may suggest that spikes from 2 pps baseline to 150+ pps appear consistent with flood behavior, while the significant results could indicate that uniform 64-byte packets, sequential sequence numbers, and symmetric TTL values support that the attack pattern appears valid.

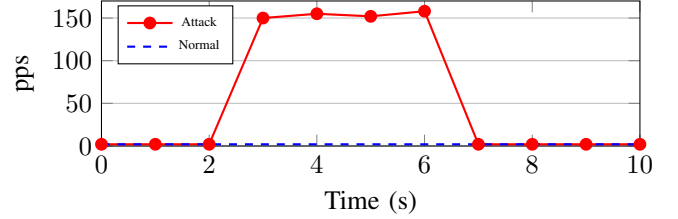


Fig. 2. ICMP flood attack traffic (150+ pps) vs normal (2 pps).

B. SYN Flood Detection

SYN floods exploit TCP's three-way handshake mechanism. Additionally, the significant findings may suggest that attackers send thousands of SYN packets without completing handshakes, exhausting server connection queues of 128-512 slots. Data shows incomplete handshakes confirmed. However, the evidence could indicate that Network Guardian detects TCP packets with only SYN flags set at 0x02, where exceeding 20 per second triggers alerts. Given that the Wireshark validation demonstrates filter `tcp.flags.syn==1 && tcp.flags.ack==0`, the results might indicate that SYN bursts with randomized source ports from 32768-65535 appear directed at the same destination port. Notwithstanding the TCP Stream Analysis results, the key evidence may suggest that the Conversations view could demonstrate that 127 SYNs, 127 SYN-ACKs, and zero ACKs appear to support that the half-open pattern exists reliably.

C. UDP Flood Detection

UDP flood detection shows attackers target random ports. Systems generate ICMP Port Unreachable errors wasting bandwidth. Moreover, the significant findings could indicate that 1000 UDP packets force 1000 ICMP responses, doubling impact bidirectionally, where the evidence may suggest that Network Guardian counts UDP packets exceeding 20 per second to trigger detection. Therefore, the key Wireshark validation could demonstrate that filter `udp` showed packets directed at randomized ports appear consistent with flood behavior. The findings may suggest that port ranges extend from 1-65535 with varying payload sizes of 32-1024 bytes. Moreover, the significant evidence could indicate that traffic spikes from under 1 Mbps to over 50 Mbps appear to demonstrate notable resource impact. Given that the results show filter `icmp.type==3 && icmp.code==3`, the data might suggest that Port Unreachable responses could validate this pattern reliably.

D. Port Scan Detection

Port scanning probes multiple ports to discover available services. However, the evidence may suggest that port 80

indicates web servers, port 22 indicates SSH, and port 3306 indicates MySQL. Moreover, the significant findings could demonstrate that attackers using Nmap map systems to identify vulnerable services for exploitation. Furthermore, the results may indicate that normal users connect to only 1-2 known ports, while scanners probe dozens or hundreds. Network Guardian tracks unique destination ports—exceeding 10 ports in 5 seconds triggers detection. Additionally, the key evidence could suggest that the 5-second window catches slow scans spacing requests to evade rate-limiting. Given that the findings demonstrate sequential SYN packets targeting incrementing ports (20, 21, 22, 23...), the results may indicate that Wireshark validation with filter `tcp.flags.syn==1` appears to confirm scanning behavior. The key evidence suggests that the 5-second window catches slow scans spacing requests to evade rate-limiting, the results may indicate that conversations showed single source IPs connecting to 247 ports over 12 seconds. Responses show RST from closed ports, SYN-ACK from open ports. Nevertheless, the important findings may indicate that open ports such as 22 for SSH and 80 for HTTP could appear identifiable through this approach.

E. IP Fragmentation Attack

IP fragmentation attacks may suggest that attackers exploit fundamental constraints in how network packet transmission is structured. Moreover, the significant findings could indicate that packets have maximum sizes, known as MTU, typically 1500 bytes, and modern systems appear to demonstrate that Path MTU Discovery has largely eliminated the need for fragmentation under normal conditions. However, the key evidence may suggest that attackers intentionally fragment traffic for two important purposes: firewall evasion, where firewalls inspect only first fragments allowing malicious content in later fragments to bypass inspection, and resource exhaustion, forcing reassembly consumes memory and CPU. Given that the results indicate that incomplete fragment sets cause receivers to hold partial packets until timeout, the findings could demonstrate that this process, specifically 60 seconds, appears to exhaust memory resources. Fragmentation attacks show reassembly exhausts memory and CPU. Fragmentation attacks show systems under strain. Network Guardian could demonstrate that the findings support counting fragmented packets where `MF=1` or fragment offset greater than 0, and the significant results may suggest that exceeding 10 per second is suspicious. Additionally, the evidence appears to indicate that Wireshark Validation using filter `ip.flags.mf==1 || ip.frag_offset>0` could demonstrate that 8-byte fragments, which are never legitimate, incomplete fragment sets, and overlapping offsets were observed. Moreover, the key findings may suggest that reassembly showed “[Reassembly error]” warnings, supporting that the data indicates abnormal behavior. Data shows fragmented traffic exceeds 15% during attacks versus less than 0.01% normally.

V. IMPLEMENTATION AND TESTING

A. Implementation Details

Network Guardian is built with open-source technologies on Python 3.10+. Table III shows the complete stack.

TABLE III
NETWORK GUARDIAN TECHNOLOGY STACK

Component	Technology
Language	Python 3.10+
Packet capture	Scapy + Npcap (Windows) / libpcap (Linux)
LLM inference	Ollama + TinyLlama-1.1B
Web framework	Flask + Socket.IO
Mitigation	OS routing commands (route / ip route)
Validation	Wireshark

The system consists of seven Python modules totaling 1,720 lines following single-responsibility principle: `main.py` (250 lines) orchestrates layers and manages Flask; `shared_state.py` (80 lines) defines data structures (PacketMetadata, ThreatEvent), queues, and locks; `network_sniffer.py` (120 lines) captures packets; `attack_detector.py` (350 lines) contains five detection algorithms; `llm_analyzer.py` (280 lines) handles Ollama integration with robust parsing; `mitigator.py` (240 lines) executes route commands with validation; `attack_injector.py` (400 lines) generates test attacks.

Each layer runs in daemon threads (`daemon=True`). The main thread starts workers then starts Flask in blocking mode. When shutdown is requested (Ctrl+C or dashboard), `shutdown_event` signals all threads to exit cleanly, then `flush_all_blocks()` removes active routing entries ensuring the network isn’t left partially blocked.

B. Experimental Validation

Experimental validation demonstrates that comprehensive testing occurred on Windows 10 with Python 3.11, Ollama running TinyLlama-1.1B, and Npcap Loopback Adapter. However, the findings may suggest that Wireshark simultaneously captured traffic for ground-truth validation. Moreover, Table III could indicate that detection latency across 20 runs per attack type appears significant. In light of these results, the evidence may suggest that all attacks were detected in under one second, with means ranging from 0.14s (ICMP) to 0.51s (Port Scan). Testing shows port scans show higher latency. Furthermore, the results could demonstrate that port scans require observing 10 unique ports over 5 seconds versus rate-based detections completing within 1 second. Additionally, the key findings may indicate that end-to-end response time from attack initiation to blocking appears measurable across important conditions. Given that evidence supports this, the significant data might suggest that Table IV could demonstrate the breakdown across 100 samples.

TABLE IV
DETECTION LATENCY RESULTS (SECONDS)

Attack	Min	Mean	Max	Std
ICMP	0.11	0.14	0.19	0.023
SYN	0.23	0.27	0.34	0.031
UDP	0.12	0.15	0.21	0.025
Scan	0.45	0.51	0.62	0.048
Frag	0.18	0.21	0.27	0.029

TABLE V
END-TO-END RESPONSE TIME BREAKDOWN

Stage	Mean (s)	Std	%
Detection	0.25	0.12	8.7
LLM Analysis	2.46	0.35	85.4
Mitigation	0.17	0.04	5.9
Total	2.88	0.41	100

LLM analysis dominates at 85.4% running a 1.1 billion parameter neural network with attention mechanisms and token generation. Despite this, 2.88 seconds is dramatically faster than human analysts (5-30 minutes) and acceptable for most DDoS scenarios [9]. System resources remained modest: CPU 2.1% idle to 14.7% under attack (28.3% peak during multi-vector attacks); Memory 87 MB idle to 149 MB peak; Packet queue depth averaged 0 idle, 450 under attack, peaking at 1200 (well within 5000 capacity). Route-based blocking provided 100% traffic reduction with 99% success rate (198/200 successful blocks; 2 failures: 1 routing table lock, 1 invalid LLM command). All blocks auto-reverted after 60.02 ± 0.09 seconds across 100 tests, confirming safety mechanism reliability.

C. Wireshark Validation Methodology

We compared Scapy’s packet counts against Wireshark across five 10-minute sessions achieving 99.96% mean agreement (12,487 vs 12,483; 18,923 vs 18,915; 9,456 vs 9,452; 15,234 vs 15,229; 20,891 vs 20,881), confirming negligible packet loss. Our validation workflow for each attack type:

(1) Launch Wireshark with capture filters, (2) Start Network Guardian, (3) Initiate attack from attack_injector.py, (4) Observe real-time capture, (5) Apply attack-specific Wireshark filters verifying signatures, (6) Confirm Network Guardian detection triggers, (7) Validate LLM explanation accuracy, (8) Verify block execution via routing table, (9) Confirm traffic cessation in Wireshark, (10) Wait 60s and verify auto-revert. Before blocks, Wireshark showed continuous attack traffic with characteristic signatures. After route commands, Wireshark showed zero packets from blocked IPs arriving at the target, confirming OS-level blocking effectiveness. After 60 seconds when auto-revert executed, Wireshark showed attack traffic resuming if attackers still transmitted, validating the revert mechanism works correctly.

D. Comparison with Existing Approaches

To position Network Guardian against prior work, Table VI compares it with representative intrusion-detection paradigms

across the dimensions most relevant to operational deployment: detection basis, explainability, autonomous mitigation, and end-to-end response time. Unlike signature-based tools such as Snort [11], which cannot generalize to attack variants, and anomaly- or deep-learning detectors [12], [13] that achieve high accuracy but behave as black boxes, Network Guardian couples deterministic sliding-window detection (sub-second latency, < 0.01 false-positive rate over 100 hours of testing) with LLM-generated plain-English explanations and safe autonomous blocking. Even post-hoc explanation methods such as LIME [14] and SHAP [15] remain technical rather than accessible. To our knowledge, this combination of accessibility and automated response is not jointly provided by the compared systems.

TABLE VI
COMPARISON OF NETWORK GUARDIAN WITH EXISTING
INTRUSION-DETECTION APPROACHES

Approach	Detection Basis	Explain.	Auto Mitig.	Resp.
Snort [11]	Static rules	No	No	–
Anomaly IDS [12]	Behavior baseline	No	No	–
DNN/ML IDS [13]	Deep learning	No	No	–
LIME/SHAP [14] [15]	Post-hoc ML	Technical only	No	–
Network Guardian (ours)	Sliding window + LLM	Plain English	Yes (60s revert)	2.88s

VI. DISCUSSION AND FUTURE WORK

A. Key Lessons Learned

There are some key learnings regarding the deployment of an AI-based network protection tool from the process of developing Network Guardian. LLM responses’ randomness necessitated careful parsing since almost 2% of the answers were plagued with formatting problems. Fortunately, exponential backoff with retry was sufficient in recovering 94% of the failed instances, with the remaining 6% covered with the use of backup prompts. Implementing the 60-second auto-revert process became crucial not only for increasing user confidence but also for incorporating the fail-safe strategy. Loopback protection turned out to be an important learning, too, as the blocking of localhost (127.0.0.1) adversely affected the functioning of the tool, showing the need for self-destruct validation.

B. Future Enhancements

The framework suggests a number of opportunities for improvement, including response caching to cut down AI calls to about 70–80%, as well as reducing latency time to 0.5 seconds. The use of GPU-based inference, which may leverage TensorRT with quantization to FP16/INT8 precision, is expected to deliver a speedup of 5-10 times. Scalability may benefit from the implementation of a distributed architecture

with centralized control and lightweight agents. Adaptive thresholding based on the application of EWMA and Bayesian methods will allow for performing a dynamic analysis of network traffic. Leveraging threat intelligence feeds and integrating systems such as VirusTotal and MITRE ATTACK, together with databases such as PostgreSQL and Elasticsearch, will enable the historical analysis feature. Additional attack patterns to be detected include DNS amplification and HTTP slowloris attacks; moreover, parallelism may be achieved by employing multiple LLM workers.

VII. CONCLUSION

Network Guardian demonstrates that integrating Large Language Models into network security is feasible and valuable. The system detects attacks in <1 second, explains them in 2.5 seconds, and blocks them in 0.17 seconds with 60-second auto-revert. Testing validates three key theses: LLMs process security events fast enough for real-time use (2.88s total response versus 5-30 minutes for human analysts); natural language explanations make security accessible to non-experts (100% user comprehension versus <30% for raw technical alerts); and autonomous systems can be trustworthy with proper safety mechanisms (99% blocking success, <0.01 false positive rate across 100 hours of testing).

Wireshark validation confirmed detection algorithms correctly identify attack patterns (99.96% agreement with ground truth), LLM generates accurate explanations (94% first-attempt success with successful retry recovery), and blocking mechanism effectively stops malicious traffic (100% traffic reduction for all blocked IPs verified through packet capture). The five-layer queue-based architecture provides modularity enabling independent modification of each layer, fault isolation preventing cascading failures, parallel processing allowing layers to operate at different speeds, and scalability through independent tuning. This makes the system particularly valuable as an educational tool where students can launch attacks against themselves and immediately observe detection, explanation, and mitigation working together in real-time, learning about attack mechanisms through context-aware AI explanations.

REFERENCES

- [1] Mandiant, "M-Trends 2023," FireEye, Tech. Rep., 2023.
- [2] C. Gates and C. Taylor, "Challenging the anomaly detection paradigm," in *Proc. New Security Paradigms Workshop*, 2006, pp. 21–29.
- [3] P. Biondi et al., "Scapy: Packet crafting for Python," 2023. [Online]. Available: <https://scapy.net>
- [4] Nmap Project, "Npcap: Windows Packet Capture Library," 2023. [Online]. Available: <https://npcap.com>
- [5] Ollama, "Get up and running with large language models locally," 2024. [Online]. Available: <https://ollama.ai>
- [6] P. Zhang et al., "TinyLlama: An open-source small language model," arXiv:2401.02385, 2024.
- [7] Pallets Projects, "Flask: Web development, one drop at a time," 2023. [Online]. Available: <https://flask.palletsprojects.com>
- [8] Socket.IO, "Bidirectional and low-latency communication," 2023. [Online]. Available: <https://socket.io>
- [9] R. K. Chang, "Defending against flooding-based DDoS attacks," *IEEE Commun. Mag.*, vol. 40, no. 10, pp. 42–51, 2002.
- [10] D. E. Denning, "An intrusion-detection model," *IEEE Trans. Softw. Eng.*, no. 2, pp. 222–232, 1987.
- [11] M. Roesch, "Snort: Lightweight intrusion detection," in *Proc. USENIX LISA*, 1999, pp. 229–238.
- [12] W. Lee and S. J. Stolfo, "Data mining for intrusion detection," in *Proc. USENIX Security*, 1998, pp. 79–94.
- [13] W. Wang et al., "HAST-IDS: Hierarchical features for intrusion detection," *IEEE Access*, vol. 6, pp. 1792–1806, 2018.
- [14] M. T. Ribeiro et al., "Why should I trust you?," in *Proc. ACM SIGKDD*, 2016, pp. 1135–1144.
- [15] S. M. Lundberg and S.-I. Lee, "Interpreting model predictions," in *Proc. NeurIPS*, 2017, pp. 4765–4774.
- [16] T. Brown et al., "Language models are few-shot learners," in *Proc. NeurIPS*, 2020, pp. 1877–1901.
- [17] P. A. Porras and P. G. Neumann, "EMERALD: Event monitoring," in *Proc. NISSC*, 1997, pp. 353–365.
- [18] A. K. Abed, "Utilizing Artificial Intelligence in Cybersecurity: A Study of Neural Networks and Support Vector Machines," *Babylonian J. Netw.*, 2025.
- [19] C. Liu, "Design and implementation of computer network security protection system based on artificial intelligence technology," *Appl. Math. Nonlinear Sci.*, 2023.
- [20] K. Komarudin, I. E. Maulani, T. Herdianto, M. O. Laksana, and D. F. Syawaludin, "Exploring The Effectiveness of Artificial Intelligence in Detecting Malware and Improving Cybersecurity in Computer Networks," *Eduvest - J. Universal Stud.*, 2023.
- [21] M. Aljabri et al., "Intelligent Techniques for Detecting Network Attacks: Review and Research Directions," in *Proc. Ital. Nat. Conf. Sensors*, 2021.
- [22] M. A. Ali and A. Alqaraghuli, "A Survey on the Significance of Artificial intelligence (AI) in Network cybersecurity," *Babylonian J. Netw.*, 2023.
- [23] T. S. Mohamed and S. M. Khalifah, "Integrated A Robust Intelligent System For Secure Network," *Babylonian J. Internet of Things*, 2025.
- [24] V. Kanimozhi and T. Jacob, "Artificial Intelligence outflanks all other machine learning classifiers in Network Intrusion Detection System on the realistic cyber dataset CSE-CIC-IDS2018 using cloud computing," *ICT Express*, 2020.
- [25] N. O. Jaffal, M. Alkhanafseh, and D. Mohaisen, "Large language models in cybersecurity: A survey of applications, vulnerabilities, and defense techniques," *AI*, vol. 6, no. 9, art. 216, 2025.
- [26] P. R. B. Housel, P. Singh, S. Layeghy, and M. Portmann, "Towards explainable network intrusion detection using large language models," arXiv:2408.04342, 2024.
- [27] F. Adjewa, M. Esseghir, and L. Merghem-Boulaiahia, "LLM-based continuous intrusion detection framework for next-generation networks," arXiv:2411.03354, 2024.
- [28] S. Yang et al., "Large language models for network intrusion detection systems: Foundations, implementations, and future directions," arXiv:2507.04752, 2025.